

Overview of Recurrent neural network

Subject: Recurrent neural network

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

1.

Global optimization methods Training the weights in a neural network can be modeled as a non-linear global optimization problem. A target function can be formed to evaluate the fitness or error of a particular weight vector as follows: First, the weights in the network are set according to the weight vector. Next, the network is evaluated against the training sequence. Typically, the sum-squared difference between the predictions and the target values specified in the training sequence is used to represent the error of the current weight vector. Arbitrary global optimization techniques may then be used to minimize this target function. The most common global optimization method for training RNNs is genetic algorithms, especially in unstructured networks. Initially, the genetic algorithm is encoded with the neural network weights in a predefined manner where one gene in the chromosome represents one weight link. The whole network is represented as a single chromosome. The fitness function is evaluated as follows:

2.

Each weight encoded in the chromosome is assigned to the respective weight link of the network. The training set is presented to the network which propagates the input signals forward. The mean-squared error is returned to the fitness function. This function drives the genetic selection process. Many chromosomes make up the population; therefore, many different neural networks are evolved until a stopping criterion is satisfied. A common stopping scheme can be:

3.

In artificial neural networks, recurrent neural networks (RNNs) are designed for processing sequential data, such as text, speech, and time series, where the order of elements is important. Unlike feedforward neural networks, which process inputs independently, RNNs utilize recurrent connections, where the output of a neuron at one time step is fed back as input to the network at the next time step. This enables RNNs to capture temporal dependencies and patterns within sequences. The fundamental building block of RNN is the recurrent unit, which maintains a hidden state—a form of memory that is updated at each time step based on the current input and the previous hidden state. This feedback mechanism allows the network to learn from past inputs and incorporate that knowledge into its current processing. RNNs have been successfully applied to tasks such as unsegmented, connected handwriting recognition, speech recognition, natural language processing, and neural machine translation. However, traditional RNNs suffer from the vanishing gradient problem, which limits their ability to learn long-range dependencies. This issue was addressed by the development of the long short-term memory (LSTM) architecture in 1997, making it the standard RNN variant for handling long-term dependencies. Later, gated recurrent units (GRUs) were introduced as a more

computationally efficient alternative. In recent years, transformers, which rely on self-attention mechanisms instead of recurrence, have become the dominant architecture for many sequence-processing tasks, particularly in natural language processing, due to their superior handling of long-range dependencies and greater parallelizability. Nevertheless, RNNs remain relevant for applications where computational efficiency, real-time processing, or the inherent sequential nature of data is crucial.

4.

Gated recurrent unit (GRU), introduced in 2014, was designed as a simplification of LSTM. They are used in the full form and several further simplified variants. They have fewer parameters than LSTM, as they lack an output gate. Their performance on polyphonic music modeling and speech signal modeling was found to be similar to that of long short-term memory. There does not appear to be particular performance difference between LSTM and GRU.

5.

Apache Singa Caffe: Created by the Berkeley Vision and Learning Center (BVLC). It supports both CPU and GPU. Developed in C++, and has Python and MATLAB wrappers. Chainer: Fully in Python, production support for CPU, GPU, distributed training. Deeplearning4j: Deep learning in Java and Scala on multi-GPU-enabled Spark. Flux: includes interfaces for RNNs, including GRUs and LSTMs, written in Julia. Keras: High-level API, providing a wrapper to many other deep learning libraries. Microsoft Cognitive Toolkit MXNet: an open-source deep learning framework used to train and deploy deep neural networks. PyTorch: Tensors and Dynamic neural networks in Python with GPU acceleration. TensorFlow: Apache 2.0-licensed Theano-like library with support for CPU, GPU and Google's proprietary TPU, mobile Theano: A deep-learning library for Python with an API largely compatible with the NumPy library. Torch: A scientific computing framework with support for machine learning algorithms, written in C and Lua.

6.

RNNs come in many variants. Abstractly speaking, an RNN is a function f_{θ} of type $(x_t, h_t) \mapsto (y_t, h_{t+1})$, where

7.

Machine translation Robot control Time series prediction Speech recognition Speech synthesis Brain-computer interfaces Time series anomaly detection Text-to-Video model Energy forecasting Rhythm learning Music composition Grammar learning Handwriting recognition Human action recognition Protein homology detection Predicting subcellular localization of proteins Several prediction tasks in the area of business process management Prediction in medical care pathways Predictions of fusion plasma disruptions in reactors (Fusion Recurrent Neural Network (FRNN) code)

8.

Stacked RNN A stacked RNN, or deep RNN, is composed of multiple RNNs stacked one above the other. Abstractly, it is structured as follows Layer 1 has hidden vector h_1, t , parameters θ_1 , and maps $f_{\theta_1} : (x_0, t, h_1, t) \mapsto (x_1, t, h_1, t+1)$. Layer 2 has hidden

vector $h_{2,t}$, parameters θ_2 , and maps $f_{\theta_2} : (x_1, t, h_{2,t}) \mapsto (x_2, t, h_{2,t+1})$ Layer n has hidden vector $h_{n,t}$, parameters θ_n , and maps $f_{\theta_n} : (x_{n-1}, t, h_{n,t}) \mapsto (x_n, t, h_{n,t+1})$. Each layer operates as a stand-alone RNN, and each layer's output sequence is used as the input sequence to the layer above. There is no conceptual limit to the depth of stacked RNN.